

ENEL712 Embedded Systems Design

Complete Exam Reference Booklet

All 12 Weeks · 24 Lectures · Exam Questions Throughout

ENEL712 — Auckland University of Technology

2025

Chapter 1

Week 1 — Introduction to Embedded Systems (L1–2)

1.1 What is an embedded system?

Embedded system A purpose-built combination of hardware and software performing a *specific* function, often under constraints of cost, power, size, reliability, and timing.

The three-verb model captures every embedded system:

Measure → Compute → Actuate

Real examples: washing machine, car ABS, smart meter, medical infusion pump, laser cutter controller.

Key concept

An embedded system is **not** a general-purpose computer. It runs one application, interacts directly with hardware, and is often invisible to the end user.

1.2 Integrated circuits and microcontrollers

Integrated circuit (IC) A device containing billions of transistors fabricated on silicon at the μm or nm scale.

Microcontroller (MCU) A *computer on a chip* — CPU + Flash + RAM + peripherals (timers, ADC, UART, GPIO) all in one IC. Reduces off-chip components, cost, and size.

1.2.1 Basic requirements to run an MCU

- DC supply (typically 3.3–5 V)
- Programmer/debugger (ISP, JTAG, or USB bootloader)
- Clock source (crystal or internal RC oscillator)
- PCB or breadboard with decoupling capacitors

1.3 When NOT to use an MCU

Use **discrete analogue/digital logic** (e.g. 555 timer, OpAmp) when the task is mathematically trivial. Use an **FPGA** when true parallel hardware processing is required. Use an **MPU** when you need a full OS, large memory, or a complex UI.

Key concept

A 32-bit MCU does not always consume more power than 8-bit. If it finishes tasks $10\times$ faster and sleeps, $E = P \times t$ may be lower overall.

1.4 Bit-width

The **bit width** (8 / 16 / 32 bit) describes the width of the ALU, registers, and internal data buses. More bits generally means: higher performance, larger addressable memory, richer peripheral set.

Exam question + model answer

Q: Explain what an embedded system is and why microcontrollers are preferred over microprocessors.

A: An embedded system is a purpose-built hardware/software combination performing a specific function under constraints. MCUs are preferred because they integrate CPU, memory and peripherals on one chip, reducing cost, PCB complexity, power, and development time. MPUs require external RAM, peripherals, and power rails, making them suitable only when high performance or a full OS is needed.

Exam question + model answer

Q: Give two situations where you would *not* choose a microcontroller.

A: (1) When the task is so simple a 555 timer or OpAmp suffices — no programmable logic needed, lower cost. (2) When the system requires Linux, multi-GB RAM, Gigabit Ethernet, or a high-resolution GUI — an MPU is more appropriate.

Chapter 2

Week 2 — MCU Operation and CPU Architecture (L3–4)

2.1 Power-on sequence

1. Supply voltage stabilises.
2. Oscillator stabilises (start-up delay, typically ms).
3. Peripherals reset to default states (pins as inputs, timers stopped).
4. Special Function Registers (SFRs) initialised.
5. Program Counter set to zero (reset vector).
6. First instruction fetched; CPU begins Fetch–Decode–Execute.

Common pitfall

If the CPU starts before supply voltage is stable, the first instruction may be misread. This is why power-on reset delay matters.

2.2 Fetch–Decode–Execute cycle

1. **Fetch:** read instruction from Flash into Instruction Register (IR).
2. **Decode:** Instruction Decoder converts machine code to control signals.
3. **Execute:** ALU performs the operation; result stored in register or RAM.

The AVR achieves **1 MIPS per MHz** because most instructions are single-cycle (RISC) and fetch/execute are pipelined.

2.3 Key CPU components

Component	Role
Program Counter (PC)	Holds address of <i>next</i> instruction; increments after each fetch; changes on branches, calls, interrupts, and reset.
Instruction Register (IR)	Holds the <i>current</i> fetched instruction during decode/execute; allows pipelining.
Instruction Decoder (ID)	Converts the IR contents to control signals for the ALU, registers, and memory.

ALU	Performs add, subtract, AND/OR/XOR, shift, complement; has direct access to all 32 general-purpose registers.
General-Purpose Registers	Fast on-chip storage (32×8 -bit on AVR, last 6 double as 16-bit). Single-cycle access.
Status Register (SREG)	Flags: Zero (Z), Carry (C), Negative (N), Overflow (V), and Global Interrupt Enable (I).

2.4 Pipelining

While instruction n **executes**, instruction $n + 1$ is **fetched**. Possible because Harvard architecture gives separate instruction and data buses.

Pipeline flush: if a branch is taken, the pre-fetched instruction is wrong and must be discarded. This is why branches cost extra cycles and lower effective MIPS.

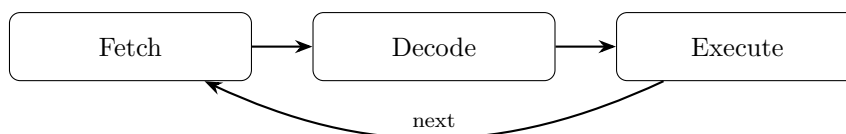


Figure 2.1: Conceptual fetch–decode–execute cycle.

2.5 RISC vs CISC

	RISC (AVR, ARM)	CISC (8051, x86)
Instructions	Simple, mostly single-cycle	Complex, multi-cycle
Code density	Larger code	Denser code
Hardware	Simpler, fewer transistors	More complex
Pipelining	Easy (uniform timing)	Harder
Power	Lower	Higher

Exam question + model answer

Q: Describe the roles of PC, IR, ALU, and SREG.

A: PC holds the address of the next instruction to fetch; it increments sequentially and changes non-sequentially on branches/calls/interrupts. IR holds the fetched instruction while it is decoded and executed, enabling pipelining. The ALU performs arithmetic and logic operations on register operands and writes results back to registers or memory. The SREG stores result flags (Z, C, N, V) used by conditional branches, and the I-bit globally enables/disables interrupts.

Chapter 3

Week 3 — Microcontroller Memory and Clocking (L5–6)

3.1 Harvard vs Von Neumann architecture

	Harvard	Von Neumann
Memory buses	Separate (instruction + data)	Shared
Pipelining	Enabled	Limited (bottleneck)
Speed	Faster (concurrent access)	Slower
Complexity	Higher	Lower
MCU examples	AVR, ARM Cortex	8051

Von Neumann Bottleneck The CPU must wait for the single shared bus to be free before it can fetch the next instruction, limiting performance.

The AT90USB mitigates this with Harvard architecture: it fetches an instruction from Flash *while* reading a variable from SRAM on the data bus.

3.2 The three memory types

Memory	Type	Key facts
Flash	Non-volatile program	Stores firmware code and constant tables. Read-only during normal operation. AT90USB: 128 KB. Endurance $\approx 100\,000$ write/erase cycles.
SRAM	Volatile data	Variables, stack, buffers. Erased on power-off. AT90USB: 8 KB. No refresh needed (static).
EEPROM	Non-volatile data	Byte-addressable. Suited to settings/calibration that must persist. Slow writes, finite endurance ($\approx 100\,000$ cycles). AT90USB: 4 KB.

Common pitfall

Never write to EEPROM in a fast loop. Each byte has limited endurance; write only when settings genuinely change.

3.3 C memory layout in SRAM

Low address		Registers / I/O (0x0000{0x00FF})	
		.data initialised globals	
		.bss uninitialised globals	
		Heap (grows up) → rarely used	
		← Stack (grows down)	
High address		Stack top (SP)	

Stack pointer (SP): 16-bit register pointing to next free stack location. Decrements on PUSH, increments on POP. Stores return addresses and local variables automatically.

Key concept

Heap (dynamic allocation with `malloc`) is usually avoided in embedded systems because of fragmentation risk, non-deterministic timing, and the danger of heap/stack collision in fixed SRAM.

Exam question + model answer

Q: Why is recursion dangerous in embedded systems?

A: Each recursive call pushes a return address (and possibly registers) onto the stack. With limited SRAM and no stack overflow protection, deep recursion quickly exhausts the stack, overwriting heap or global variables and causing unpredictable crashes.

3.4 Clock sources

Source	Accuracy	Typical use
External crystal (8/16 MHz)	$\pm 0.01\text{--}0.1\%$	UART, PWM, SPI, I ² C — any timing-sensitive application.
Internal RC oscillator	$\pm 10\%$	Flashing an LED only. Not suitable for communications.
Low-freq crystal (32.768 kHz)	Good	Real-time clocks; stays active in low-power sleep modes.
External clock input	Varies	When another device provides the clock.

Clock prescaler: $f_{CPU} = f_{OSC}/N$. By default the AT90USB applies a prescaler of 8, so an 8 MHz crystal gives $f_{CPU} = 1$ MHz.

Exam question + model answer

Q: Why is the internal RC oscillator unsuitable for UART communication?

A: UART relies on precise bit timing. The internal RC oscillator has $\pm 10\%$ frequency error; this causes baud rate mismatch between transmitter and receiver, leading to framing errors.

and corrupted data. A crystal or resonator is required.

Chapter 4

Week 4 — Advanced MCUs and Microprocessors (L7–8)

4.1 When 8-bit is not enough

Upgrade beyond 8-bit when you need:

- heavier arithmetic (trigonometry, control loops, DSP),
- higher timing precision or PWM resolution,
- complex communications (USB, Gigabit Ethernet, crypto),
- more RAM/Flash (networking stacks, large buffers),
- potential for future feature growth.

Key concept

State *what* the bottleneck is. “Not enough performance” is too vague. Specify: CPU cycles, RAM, Flash, peripheral count, timing precision, or bus bandwidth.

4.2 Enhanced MCU families

Atmel XMEGA: 8-bit data bus, 16-bit ALU. Adds DMA (moves data without CPU), Event System (peripheral-to-peripheral triggers), AES hardware.

DSP/MCU hybrids (TI C2000): 32-bit, single-cycle 32×32 hardware multiply. Closes control loops at 20 kHz+ versus 1 kHz on a basic MCU.

4.3 ARM architecture

ARM Holdings licenses the core IP; manufacturers (Atmel SAM, STM32, Qualcomm, Apple) add their own peripherals and memory.

ARM success factors: RISC efficiency (fewer transistors $\Rightarrow$ lower power), dominant in smartphones and embedded (95 % of smartphones in 2010), strong ecosystem.

4.4 MCU vs MPU

	MCU	MPU
Memory	On-chip Flash + SRAM	External DRAM + NAND Flash (GBs)
OS	Bare-metal or RTOS	Full Linux / Android
Power	Low (sleep modes, μA)	Higher (>400 DMIPS for Linux)
PCB	Simple	Complex (6+ layers, impedance-matched DDR traces)
Power supply	Single rail (3.3/5 V)	Multiple rails (1.2 V, 1.8 V, 3.3 V) with sequencing
Boot time	Fast (<1 ms typical)	Seconds (OS boot)
Use when	Real-time control, low-power, simple/no UI	Colour UI, high-speed networking, file systems

Exam question + model answer

Q: A designer needs a colour touchscreen, Ethernet, and months of log storage. MCU or MPU?

A: MPU. A colour UI with Ethernet and long-term storage benefits heavily from an OS with built-in TCP/IP, USB, and file-system drivers. An MCU solution would require external memory, complex custom stacks, and significant additional development time.

Chapter 5

Week 5 — Embedded Digital Communications A (L9–10)

5.1 Parallel vs serial

	Parallel	Serial
Wires	Many (8, 16, 32...)	Few (1–4)
Speed (low clock)	Higher throughput	Lower
Speed (high clock)	Limited by crosstalk/skew	Higher (GHz possible)
Distance	Short	Long
Example	Old IDE drives	SATA, USB, SPI

At high clock frequencies, parallel buses suffer from **clock skew** (bits arrive at different times) and **crosstalk** (EMI between adjacent wires). Serial avoids both, which is why SATA replaced IDE.

5.2 Asynchronous serial: RS-232 / UART

Frame format: [Start] [8 data bits] [Parity] [Stop] — total 10 bits per byte for 8N1.

$$\text{Bytes/s} \approx \frac{\text{Baud}}{10} \Rightarrow 9600 \text{ baud} \approx 960 \text{ bytes/s}$$

Start (0)	Data bits D0–D7	Parity	Stop (1)
1 bit	8 bits	1 bit (opt.)	1–2 bits

UART 8N1 frame. Both sides must agree on baud rate and format.

TTL vs RS-232: MCU UARTs use TTL levels (0–3.3 V or 0–5 V). True RS-232 uses ± 12 V inverted logic. Use an FTDI chip or MAX232 level shifter to interface with PCs.

Common formats: 9600 8N1, 38400 8N1, 115200 8N1.

Common pitfall

Baud is bits per second, not bytes per second. Always divide by the frame length (10 for 8N1) to get byte throughput.

5.3 RS-422 and RS-485

Same UART framing as RS-232 but use **differential signalling** on twisted pair.

- RS-422: range >1 km, multi-drop (1 Tx, 10 Rx).
- RS-485: multi-point network (32 Tx/Rx), common in industrial systems.
- Half-duplex (2 wires); add termination resistors at cable ends to prevent reflections.

5.4 SPI (Serial Peripheral Interface)

SPI Synchronous, full-duplex, 4-wire: MOSI, MISO, SCK, CS (active-low).

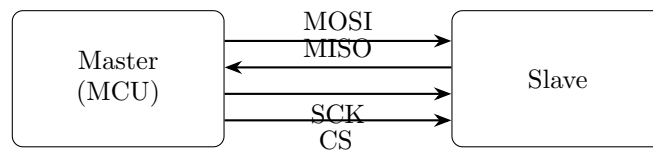


Figure 5.1: SPI: master drives clock and CS; full-duplex simultaneous Tx/Rx.

Key facts:

- Master generates SCK and selects slave with CS (one CS pin per slave).
- To read, master must *clock* (transmit dummy bytes).
- For 1 master + 3 slaves: 6 master pins (MOSI, MISO, SCK + 3×CS).
- CPOL/CPHA select idle clock level and sampling edge; master and slave must match.

Exam question + model answer

Q: A system has 1 SPI master and 3 slaves. How many pins on the master are used for SPI?

A: 6 pins: MOSI, MISO, SCK (shared by all slaves) plus 3 individual Chip Select lines.

5.5 I²C (Inter-Integrated Circuit)

I²C Two-wire synchronous bus (SCL + SDA) supporting multiple addressed devices.

Key facts:

- **Open-drain** lines — devices only pull low; **pull-up resistors** pull high.
- **Start condition:** SDA falls while SCL is high.
- **Address frame:** 7-bit address + R/ $\bar{W}$ bit + ACK from slave.
- Speeds: 100 kHz (standard), 400 kHz (fast), 1 MHz (fast-plus).
- 20 sensors: only 2 MCU pins needed (vs 23 for SPI).

Common pitfall

Forgetting pull-up resistors is the most common I²C mistake. Without them the lines can never go high and the bus hangs.

5.6 Protocol comparison

Protocol	Wires	Speed	Multi-device	Best for
UART/RS-232	2 (Tx, Rx)	Up to 1 Mbps	No (point-to-point)	Debug console, long cables
SPI	3+CS	>10 MHz	Yes (1 CS/slave)	Fast ADC, SD cards, displays
I ² C	2	100 k–1 MHz	Yes (addressed)	Many sensors, crowded PCB
CAN	2 (diff)	1 Mbps	Yes (multi-master)	Automotive, industrial

Exam question + model answer

Q: Choose UART, SPI, or I²C for: (a) fast ADC streaming, (b) 10 low-speed sensors, (c) debug console to PC.

A: (a) SPI — high speed, simple full-duplex clocking, dedicated CS. (b) I²C — only 2 wires regardless of sensor count; addressing handles all devices. (c) UART — simplest point-to-point serial, bridged to USB with FTDI.

Chapter 6

Week 6 — Embedded Digital Communications B (L11–12)

6.1 Differential signalling

Single-ended One data wire referenced to ground. Susceptible to common-mode noise. Differential Two complementary wires; receiver measures $V^+ - V^-$. Noise couples equally onto both wires (common-mode) and cancels at the receiver.

Benefits: noise immunity, lower EMI, longer distance, higher clock rates.

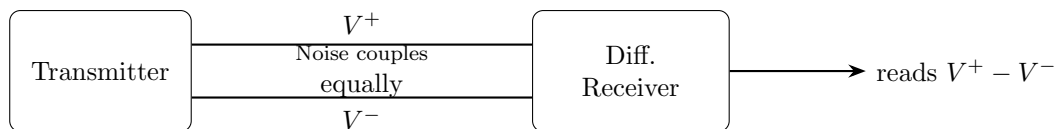


Figure 6.1: Differential link: common-mode noise is rejected at the receiver.

6.2 Distance vs data rate

Higher frequencies cause transmission-line effects: reflections, capacitance-limited slew rate, and crosstalk. Reliable distance is **inversely proportional** to data rate.

6.3 USB overview

USB Host-controlled serial bus for both power and data, tiered-star topology.

Key exam facts:

- **Host-controlled:** only the host initiates transfers; devices respond.
- Enumeration assigns addresses; descriptors describe device capabilities.
- Endpoints are the device's "pipes" (bulk, interrupt, isochronous, control).
- USB 2.0 speeds: Low-Speed (1.5 Mbps), Full-Speed (12 Mbps), Hi-Speed (480 Mbps).
- Requires a dedicated hardware controller (e.g. AT90USB has one built in).

Exam question + model answer

Q: Why is USB harder to implement than UART?

A: UART is a simple byte stream requiring only baud rate agreement. USB requires: enumeration (address assignment), descriptor tables (device identity), endpoint management (separate data pipes), packet-based protocol, host scheduling, and a dedicated hardware USB controller or software stack. The complexity is an order of magnitude higher.

6.4 Ethernet overview

Requires external MAC/PHY or MPU-class processor plus a full TCP/IP stack. Typically uses DDR/SDRAM buffers. Suitable when connecting to existing LAN infrastructure or the internet.

Chapter 7

Week 7 — Embedded Hardware Design from a Specification (L13–14)

7.1 Design workflow

1. Draw a **block diagram** (MCU, sensors, actuators, comms, power, programming header).
2. Research and select **I/O devices** (datasheets: interface, voltage, current, accuracy).
3. Compile an **I/O tally**.
4. Select the **MCU** that satisfies the tally plus memory and performance.
5. Design the **power supply**.

Key concept

Always select I/O devices *before* choosing the MCU. The MCU must fit the I/O requirements, not the other way round.

7.2 I/O tally

Category	What to count
Digital I/O	# inputs, # outputs
Analogue	# ADC channels, any DAC or PWM outputs
Serial interfaces	# UARTs, SPIs, I ² Cs, CANs
Timer/PWM channels	# PWM outputs, input capture lines
Programming	JTAG / ISP (often forgotten!)

7.3 MCU selection criteria

I/O count → memory (Flash/RAM headroom) → operating voltage → power modes → package (SMD/DIP) → cost/availability → toolchain familiarity.

7.4 Power supply design

7.4.1 Current tally

List each rail (e.g. 3.3 V, 5 V), sum active current per device, add 20 % margin.

7.4.2 Linear vs switching regulators

	Linear (LDO)	Switching (buck/boost)
Efficiency	Poor when $V_{in} \gg V_{out}$	Often >80 %
Noise	Low	High (switch noise)
Complexity	Simple	More complex
Heat	$(V_{in} - V_{out}) \times I_{load}$	Minimal

Best practice: use a **switching regulator** first (efficiency), followed by an **LDO** to clean the noise before the MCU supply.

Exam question + model answer

Q: A 9 V battery must supply 3.3 V at 250 mA. Linear or switching?

A: Switching. A linear regulator would dissipate $(9 - 3.3) \times 0.25 = 1.4$ W as heat, giving very poor efficiency and likely requiring a heatsink. A buck converter achieves >80 % efficiency, significantly extending battery life.

Chapter 8

Week 8 — Timers, Counters and PWM (L15–16)

8.1 Why hardware timers?

Software delay (`_delay_ms()`) blocks the CPU: no other processing occurs during the wait.

Hardware timer increments a register automatically at a rate set by the clock and prescaler, firing an interrupt or setting a flag when it overflows or matches a compare value. The CPU is free to do other work.

Exam question + model answer

Q: Why is using a software delay in a system that must respond to frequent user input considered poor practice?

A: Software delays consume 100 % of CPU cycles during the wait, preventing the processor from sampling inputs, updating displays, or responding to any event. A hardware timer runs independently, allowing the CPU to remain responsive and only act when the timer interrupt fires.

8.2 Timer prescaling

The prescaler divides the system clock before the timer counts:

$$f_{timer} = \frac{f_{clk}}{N} \quad \text{where } N \in \{1, 8, 64, 256, 1024, \dots\}$$

Worked example

$f_{clk} = 8 \text{ MHz}$, prescaler $N = 64$:

$$f_{timer} = \frac{8,000,000}{64} = 125 \text{ kHz} \quad \Rightarrow \quad t_{tick} = \frac{1}{125,000} = 8 \mu\text{s}$$

An 8-bit timer (256 counts) overflows every $256 \times 8 \mu\text{s} = 2.05 \text{ ms}$.

Worked example

An 8-bit timer, no prescaler, $f_{clk} = 1 \text{ MHz}$: each tick = $1 \mu\text{s}$; overflow = $256 \mu\text{s}$.

8.3 Timer modes

Mode	Use	How it works
Normal	Simple delay/overflow	Counts up; interrupt on overflow.
CTC (Clear on Compare)	Precise period	Clears when it reaches OCR; interrupt at top.
Fast PWM	Motor/LED control	Counts up to TOP (OCR or ICR), toggles output pin.
Phase-correct PWM	Smooth PWM	Counts up then down; symmetrical duty cycle.
Input Capture	Measuring pulse width	Latches timer value on signal edge for precise timing.

Timer vs Counter A **timer** increments from the internal clock (fixed interval). A **counter** increments from an external signal on a pin (e.g. counting encoder pulses).

8.4 Pulse Width Modulation (PWM)

PWM approximates an analogue voltage by switching a digital pin ON/OFF rapidly.

$$V_{avg} = V_{supply} \times D \quad \text{where } D = \frac{t_{on}}{T} \text{ is the duty cycle}$$

Worked example

$V_{supply} = 5 \text{ V}$, $D = 25 \%$: $V_{avg} = 5 \times 0.25 = 1.25 \text{ V}$. An LED appears at $\approx 25 \%$ brightness.

For the AT90USB (Timer 1, Fast PWM, ICR1 = 399, prescaler 1 at 8 MHz):

$$f_{PWM} = \frac{8,000,000}{1 \times 400} = 20 \text{ kHz}$$

OCR range 0–399; 399 = 100 % duty cycle.

Exam question + model answer

Q: In a drone flight controller, why is hardware PWM *mandatory* for motor control?

A: Controlling four motors by software bit-banging would require constant CPU attention at kHz rates, leaving no cycles for sensor reading, stability calculations, or communication. Hardware PWM generates the control signals autonomously, freeing the CPU entirely for flight-critical logic.

8.5 Timer design decision table

Need	Implementation	Impact
Simple LED blink	<code>_delay_ms()</code>	Easy, CPU blocked during delay
Precise periodic task	Timer + interrupt (CTC)	CPU free; accurate timing
Dim LED / speed motor	Hardware Fast PWM	No CPU overhead; autonomous
Measure pulse width	Input Capture mode	Hardware timestamps edges

Chapter 9

Week 9 — Interrupts and Analog-to-Digital Conversion (L17–18)

9.1 Polling vs interrupts

Polling: main loop repeatedly checks a flag. Simple, predictable. Wastes CPU; may miss very short events.

Interrupt: hardware suspends the main program, executes an ISR, then resumes. Responsive, efficient; adds concurrency complexity.

Exam question + model answer

Q: Describe a scenario where polling fails but an interrupt succeeds.

A: Capturing a $10\ \mu\text{s}$ pulse from a high-speed sensor. If the CPU is executing a 1 ms function, polling will miss the pulse entirely. An interrupt hardware-triggers the CPU to stop immediately, latch the event, and return, guaranteeing the pulse is captured.

9.2 How an interrupt works

1. Event occurs (pin edge, timer overflow, UART byte received, etc.).
2. CPU finishes the current instruction.
3. PC is pushed to the stack.
4. CPU jumps to the Interrupt Vector Table (IVT) address for this interrupt.
5. ISR executes.
6. CPU pops the PC and resumes the main program.

9.3 Three keys to enabling an interrupt

1. Enable the **peripheral interrupt bit** (e.g. OCIE1A for Timer 1 compare A).
2. Set the **Global Interrupt Enable** (I-bit in SREG, via `sei()` in C).
3. The **triggering event** must actually occur.

Common pitfall

A student configures the Timer 1 overflow interrupt bit but the ISR never fires: almost certainly because `sei()` was not called to set the global I-bit.

9.4 ISR best practices

- Keep ISRs as **short as possible**: set a flag, increment a counter, copy a byte; do real processing in the main loop.
- Use `volatile` for shared variables: without it the compiler may cache the variable in a register and not see updates from the ISR.
- Disable interrupts (`cli()/sei()`) around multi-byte critical sections in the main loop.
- Avoid `printf`, `malloc`, or any non-reentrant library function inside an ISR.

9.5 ADC fundamentals

The AT90USB has a 10-bit successive-approximation ADC (0–1023 counts for 0– V_{ref}).

$$\text{ADC resolution} = \frac{V_{ref}}{2^n - 1} \quad (n = 10 \Rightarrow \approx 4.88 \text{ mV per step at } V_{ref} = 5 \text{ V})$$

$$\text{ADC value} = \frac{V_{in}}{V_{ref}} \times (2^n - 1)$$

Worked example

$V_{ref} = 2.56 \text{ V}$, $V_{in} = 1.0 \text{ V}$, $n = 10$:

$$\text{ADC} = \frac{1.0}{2.56} \times 1023 \approx 399$$

9.6 Voltage reference (V_{ref}) selection

Option	Value	When to use
V_{CC}	5 V	General-purpose; best when signals span 0–5 V.
Internal	1.1 V or 2.56 V	Sensor outputs <1 V; improves precision.
External AREF	User-supplied	Requires accurate reference voltage source.

Common pitfall

If you select the internal 2.56 V reference in software *while* a 5 V source is connected to the AREF pin, an internal short circuit may damage the MCU's analogue hardware permanently.

9.7 ADC clock prescaler

The successive-approximation circuit needs time to settle. Keep the ADC clock between **50 kHz and 200 kHz** for full 10-bit resolution. Use the ADC prescaler register (ADPS) to divide f_{clk} .

A 10-bit conversion takes **13–25 ADC clock cycles** ($\approx 65\text{--}500\ \mu\text{s}$ at 200 kHz).

9.8 ADC multiplexing and registers

- **ADMUX:** selects channel (MUX bits), reference (REFS), and left-adjust (ADLAR).
- **ADCSRA:** enable (ADEN), start conversion (ADSC), auto-trigger (ADATE), interrupt enable (ADIE), prescaler (ADPS).

One ADC can read many pins sequentially by changing the MUX between conversions. Only one channel is converted at a time.

Exam question + model answer

Q: Why can't the ADC clock run at the full 8 MHz system frequency?

A: The successive-approximation comparator needs time between bit trials for the voltage to settle. At high speeds the settling is incomplete, causing premature decisions and incorrect bits. The ADC prescaler limits the clock to the recommended 50–200 kHz range for accurate 10-bit results.

Chapter 10

Week 10 – Embedded Software Engineering (L19–20)

10.1 Software engineering in context

Software engineering An engineering discipline covering all aspects of software production: requirements, design, implementation, validation, and evolution.

Embedded adds:

- hardware constraints (SRAM, Flash, timing, power),
- difficulty of debugging (no OS, limited visibility),
- reliability and safety requirements,
- direct sensor/actuator interfaces.

10.2 Four process activities

1. **Specification:** what must the system do? Under what constraints?
2. **Development:** design and implement the solution.
3. **Validation:** did we build the right thing? Does it meet requirements?
4. **Evolution:** how does the system change safely over time?

10.3 Requirements

Functional requirement What the system does — a behaviour or service. **Non-functional requirement** How well or under what conditions — performance, reliability, power, security.

A good requirement is: unambiguous, testable, necessary, feasible, and traceable to a test.

Common pitfall

Bad: “The device should be fast.”

Better: “The device shall update the display within 200 ms and log data at 10 Hz.”

Exam question + model answer

Q: Write one functional and one non-functional requirement for a battery-powered temperature logger.

A: FR: “The device shall measure and store a timestamped temperature sample every 10 s.”

NFR: “The device shall operate for at least 30 days on a 2000 mAh battery under normal sampling conditions.”

10.4 Software testing

Validation testing Confirms the system performs correctly for its intended use. A pass shows the system works as expected. Defect testing Deliberately tries to expose failures using edge cases, out-of-range inputs, and adversarial sequences. A pass is a test that makes the system behave incorrectly.

Testing can reveal the *presence* of errors, not their absence.

10.5 Fault-tolerant and defensive design

- **Power-On Self-Test (POST):** verify all components respond correctly at startup.
- **Checksums:** verify stored settings have not been corrupted.
- **Watchdog timer:** automatically resets the MCU if the main loop gets stuck.
- **Boundary checks:** validate all input values before use.
- **Default all unused interrupt vectors** to a safe handler to prevent runaway from noise-triggered jumps.
- **Re-write registers** periodically during normal operation, not just at startup.

Chapter 11

Week 11 — Software Architectures and Modular Design (L21–22)

11.1 Software architecture patterns

11.1.1 Polling loop

Simple; checks all inputs sequentially in a `while(1)`. Can use a timer flag to synchronise the loop period. CPU always active — wasteful for power-sensitive designs.

11.1.2 Event-driven (interrupt-driven)

Interrupts signal events; main loop may sleep between events. Most power-efficient. Adds concurrency risk: shared variables between ISR and main must be `volatile` and access must be atomic.

11.1.3 State machine

Divides behaviour into a finite set of states with defined transitions. Implemented in C as a `switch(state)` in the main loop; clear, testable, and prevents illegal states.

Listing 11.1: Minimal switch-case state machine.

```
1 typedef enum { IDLE, MEASURE, SEND } state_t;
2 state_t state = IDLE;
3 for (;;) {
4     switch (state) {
5         case IDLE:    if (timer_flag) state = MEASURE; break;
6         case MEASURE: sample = read_sensor(); state = SEND; break;
7         case SEND:    uart_send(sample); state = IDLE; break;
8     }
9 }
```

11.1.4 RTOS

OSManages resources and time-slices tasks; timing guarantees are **not** guaranteed. RTOSDesigned so every task meets its deadline deterministically. Examples: FreeRTOS, RTLinux.

Use an RTOS when tasks have strict timing requirements and polling/interrupt models become too complex.

11.2 Hardware/software partitioning

Some features can be placed in either hardware or software:

- Switch debouncing (hardware RC or software counter),
- UART/SPI/I²C (hardware peripheral vs bit-banging),
- CRC calculation (hardware accelerator vs software),
- DMA transfers (hardware vs CPU memcpy).

Use built-in hardware peripherals where available; it frees CPU cycles and improves timing accuracy.

11.3 Modular software design

ModuleA grouping of related functionality with a defined interface (API) and hidden implementation.

Design recipe:

1. Identify 3–5 modules (categories of functionality).
2. Assign one or more source files per module.
3. Define functions per source file (3–6 per file is typical).
4. Define data structures passed between modules.
5. Identify which peripherals require interrupts and which module owns them.
6. Identify reuse opportunities.

11.4 Driver levels

Level	Example	Responsibility
Low-level	<code>SPDR = byte;</code>	Direct register access.
Mid-level	<code>spi_transfer(byte)</code>	Send/receive individual bytes; hide register details.
High-level	<code>sensor_read_temp()</code>	Device-specific logic; calls mid-level.

11.5 Practical C techniques

11.5.1 Arrays and strings

Strings in C are null-terminated `char[]` arrays. Risks: buffer overflow, missing `'\0'`, heavy `printf` code size. Use `sprintf` for formatted strings on embedded systems; avoid floating-point unless absolutely needed.

11.5.2 Pointers

Pointers store memory **addresses**, not values. Useful for:

- hardware register access (`volatile uint8_t *reg = (uint8_t *)0x60;`),
- passing large arrays/structs by reference (avoids stack copy),
- pointer arithmetic (fast array indexing).

11.5.3 Structures

Group related fields. Always pass structures to functions as pointers.

```
1 typedef struct { int16_t temp_c_x100; uint32_t ts_ms; uint8_t status; }
2   sensor_record_t;
```

11.5.4 Standard library quick reference

Function	Header	Purpose
sprintf	stdio.h	Format variables into a <code>char[]</code> for serial/LCD output.
sscanf	stdio.h	Parse a string into variables (inverse of <code>sprintf</code>).
atoi/atof	stdlib.h	Convert ASCII string to integer/float.
memcpy	string.h	Copy <i>n</i> bytes between buffers.
memset	string.h	Initialise a buffer to a value (typically zero).
strcpy	string.h	Copy a null-terminated string.

Exam question + model answer

Q: Why should structures be passed by pointer rather than by value in embedded C?

A: Passing by value copies the entire struct onto the stack; on a small MCU this wastes precious SRAM and adds push/pop overhead for every call. Passing a pointer (typically 2 bytes on AVR) avoids the copy, reduces stack usage, and allows the function to modify the original data.

Chapter 12

Week 12 — DSP Fundamentals, Power Management and OOP (L23–24)

12.1 Sampling and the Nyquist theorem

Nyquist–Shannon theorem The sampling frequency f_s must be at least twice the highest frequency component f_{max} in the signal: $f_s \geq 2f_{max}$.

Aliasing When $f_s < 2f_{max}$, high-frequency components “fold” into lower frequencies, creating phantom signals that do not exist in the original.

Always use an **anti-aliasing filter** (low-pass analogue filter) before the ADC to remove frequency content above $f_s/2$.

Exam question + model answer

Q: A digital stethoscope must capture heart sounds up to 2 kHz. Minimum sampling rate? What happens at 3 kHz?

A: Minimum $f_s = 2 \times 2 \text{ kHz} = 4 \text{ kHz}$. At 3 kHz, frequencies between 1.5 kHz and 2 kHz alias into the 0–1.5 kHz range, appearing as phantom low-frequency sounds that corrupt the reading.

12.2 Quantisation and SNR

When a continuous voltage is rounded to the nearest ADC step, the rounding error is **quantisation noise**.

$$\text{SNR (dB)} \approx 6.02n + 1.76 \quad (n = \text{ADC bits})$$

Each additional bit adds $\approx 6 \text{ dB}$ (doubles the dynamic range). A 10-bit ADC: $\approx 62 \text{ dB}$; a 16-bit audio ADC: $\approx 98 \text{ dB}$.

Exam question + model answer

Q: How does upgrading from 8-bit to 12-bit ADC affect quantisation noise?

A: A 12-bit ADC has $2^4 = 16$ times more discrete levels than 8-bit. The quantisation step is $16\times$ smaller, dramatically reducing quantisation noise and improving SNR by approximately $4 \times 6 = 24 \text{ dB}$.

12.3 Digital filters

LTI system Linear Time-Invariant — the system's response to any input can be predicted from its impulse response. Enables digital filter design.

FIR filter Finite Impulse Response — uses only current and past *inputs*; always stable. **IIR filter** Infinite Impulse Response — uses current inputs *and* past *outputs* (feedback); can be unstable if coefficients are poorly chosen.

12.3.1 Moving-average (FIR low-pass)

$$y[n] = \frac{1}{2} x[n] + \frac{1}{2} x[n-1]$$

Averages current and previous sample; smooths rapid (high-frequency) fluctuations.

Exam question + model answer

Q: Why are FIR filters preferred over IIR in safety-critical embedded systems?

A: FIR filters use no feedback, so they cannot oscillate or become numerically unstable regardless of coefficient values. IIR filters use past output values on the right-hand side of the difference equation; if the coefficients are not carefully selected the filter poles can lie outside the unit circle, causing exponential growth (instability). In safety-critical contexts the guaranteed stability of FIR is worth the extra computation.

12.4 Power management and sleep modes

MCU power consumption is dominated by the active clock. Sleep modes gate clocks to unused modules.

Mode	Current (typical)	What remains active
Active	mA	Everything.
Idle	≈10–50 % of active	CPU stopped; peripherals, timers, UART still tick.
Power-down	μA	Almost nothing. Wake only via external interrupt or watchdog.
Standby	μA + oscillator	Crystal oscillator kept running for fast wake.

12.4.1 Average current with duty cycles

If the device is active for fraction D of the time:

$$I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep}$$

$$\text{Battery life (h)} \approx \frac{C_{battery} \text{ (mAh)}}{I_{avg} \text{ (mA)}}$$

Worked example

Active 1 % of time at $I_{active} = 15 \text{ mA}$; sleep at $I_{sleep} = 20 \mu\text{A}$:

$$I_{avg} = 0.01 \times 15 + 0.99 \times 0.02 \approx 0.17 \text{ mA}$$

2000 mAh battery: life $\approx 2000/0.17 \approx 11765 \text{ h} \approx \mathbf{490 \text{ days}}$.

Exam question + model answer

Q: A sensor node draws 12 mA for 50 ms every 5 s; sleeps at $30 \mu\text{A}$ otherwise. Estimate average current and battery life for 2400 mAh.

A: $D = 0.05/5 = 0.01$. $I_{avg} = 0.01 \times 12 + 0.99 \times 0.03 \approx 0.15 \text{ mA}$. Life $\approx 2400/0.15 = 16,000 \text{ h} \approx 667 \text{ days}$ (real life lower due to self-discharge and voltage cutoff).

12.4.2 Other power-saving strategies

- Clock prescaling (lower f_{CPU} when full speed not needed).
- Disable unused peripherals (ADC, SPI, UART) via Power Reduction Register (PRR).
- Avoid floating input pins (pull-ups prevent leakage current).
- Minimise radio or Wi-Fi duty cycle (dominant energy consumer in wireless designs).

12.5 Object-Oriented Programming for embedded GUI

When the PC-side control application is written in C# or C++, OOP principles improve maintainability:

EncapsulationHide the complex serial protocol details inside an `AppBoard` class. The UI calls `motor.SetSpeed(50)` without knowing the hex framing. **Abstraction**The UI developer interacts with “objects” (`Motor.Start()`) without needing the register-level hex codes. **Inheritance / Polymorphism**Extend a generic `Sensor` base class for each specific sensor type.

Exam question + model answer

Q: How does Abstraction in OOP benefit a PC-based control dashboard for an embedded device?

A: Abstraction allows the UI developer to work with high-level objects (e.g. `Heater.SetDuty(75)`) without knowing the UART packet format, hex instruction codes, or register-level details. This separates interface design from hardware implementation, enabling parallel development and easier maintenance when the protocol changes.

Quick-Reference Summary Tables

Protocol comparison

Protocol	Wires	Speed	Multi-device	Choose when
UART	2	Up to 1 Mbps	No	Debugging, long cables, PC comms
SPI	3+CS	>10 MHz	Yes (1 CS/slave)	Fast ADC, displays, SD cards
I ² C	2	100 k–1 MHz	Yes (7-bit addr)	Many sensors, crowded PCB
RS-485	2 (diff)	10 Mbps	32 nodes	Industrial multi-drop
USB	2 (diff)	Up to 480 Mbps	Enumerated	Consumer PC interface
CAN	2 (diff)	1 Mbps	Multi-master	Automotive, safety-critical

Memory types

Memory	Volatile?	Speed	Use
Flash	No	Slow write	Firmware code, constants
SRAM	Yes	Fast	Variables, stack, buffers
EEPROM	No	Slow write	Settings persisting over power cycles

Timer design

Need	Method	Notes
Simple pause	<code>_delay_ms()</code>	Blocks CPU
Periodic interrupt	CTC + ISR	CPU free between events
PWM output	Fast/Phase-correct PWM	Hardware autonomous
Pulse measurement	Input Capture	Hardware timestamps

Power estimation quick formulas

$$I_{avg} = D \cdot I_{active} + (1 - D) \cdot I_{sleep} \quad \text{Life (h)} = \frac{C_{mAh}}{I_{avg}}$$

$$\text{SNR (dB)} \approx 6.02n + 1.76 \quad f_s \geq 2f_{max} \quad (\text{Nyquist})$$

Week-by-week lecture map

Week	Lectures	Topic
1	L1–2	Intro to embedded systems, MCU overview
2	L3–4	CPU architecture, fetch–decode–execute, RISC/CISC
3	L5–6	Memory (Flash/SRAM/EEPROM), stack, clocking
4	L7–8	Advanced MCUs, ARM, MCU vs MPU
5	L9–10	Comms A: UART, SPI, I ² C, CAN
6	L11–12	Comms B: differential, USB, Ethernet
7	L13–14	Hardware design from spec, I/O tally, power supply
8	L15–16	Timers, counters, PWM
9	L17–18	Interrupts, ISR, ADC, quantisation
10	L19–20	Software engineering, requirements, testing
11	L21–22	SW architectures, modular design, C techniques
12	L23–24	DSP/Nyquist/filters, power management, OOP

Suggested external references

- Microchip AT90USB1287 Datasheet.
- Microchip “AVR Memory” and “AVR Low Power Sleep Modes” application notes.
- NXP UM10204 I²C-bus specification.
- Analog Devices: SPI interface primer; CAN implementation guide.
- USB 2.0 Specification (April 2000).
- <https://learn.sparkfun.com/tutorials/serial-communication>
- <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi>

Week 13 — Device Interface Quick Reference

This chapter provides a practical wiring and selection guide for common peripheral devices used with 5 V MCUs such as the AT90USB1287. **Intermediate device key:** Direct = wire straight to GPIO/ADC; Driver IC = needs transistor or H-bridge; Level Shift = 3.3 V↔5 V mismatch; Signal Cond. = amplifier or converter required.

Temperature and humidity sensors

Device	Voltage	Interface	Intermediate	Notes
DS18B20	3.0–5.5 V	1-Wire	Direct + 4.7 k Ω	Digital; daisy-chain many on 1 wire; 12-bit = 0.0625°C
DHT22	3.3–5 V	Single-wire	Direct + 10 k Ω	Temp + humidity; max 0.5 Hz sample rate
LM35	4–30 V	Analog	Direct	10 mV/°C; connect to ADC; no calibration needed
HTS221	1.7–3.6 V	I ² C	Level shift (5 V MCU)	3.3 V IoT sensor; address 0x5F; 2 μ A idle
BME280	1.71–3.6 V	I ² C/SPI	Level shift (5 V MCU)	Temp + humidity + pressure; 3.3 V only

Motion and distance sensors

Device	Voltage	Interface	Intermediate	Notes
HC-SR501 PIR	4.5–20 V	Digital	Direct	HIGH on motion; 3.3 V logic-compatible
HC-SR04 Ultrasonic	3.3–5 V	GPIO Trig/Echo	Direct	10 μ s trigger; measure echo width; Echo is 5 V
MPU-6050	2.375–3.46 V	I ² C	Level shift	3.3 V; INT pin for data-ready interrupt

Actuators — motors, servos, relays

Device	Drive	Interface	Intermediate	Notes
DC Motor	6–24 V	PWM + GPIO	L298N or DRV8833	NEVER connect directly to MCU; 2A H-bridge; add flyback diodes
Stepper Motor	8–35 V	STEP+DIR	A4988 or TMC2209	Driver IC manages coil current; set Vref for current limit
Servo (SG90)	4.8–6 V	PWM (50 Hz)	Direct (separate supply)	1–2 ms pulse at 50 Hz; servo power must be separate
Relay Module	5 V coil	GPIO	Direct (opto-isolated)	Active-LOW; pre-made modules have transistor driver built in
Solenoid Valve	12–24 V	GPIO	MOSFET + flyback diode	IRLZ44N; flyback diode mandatory — inductive spike kills MCU

Wireless and communication modules

Device	Voltage	Interface	Intermediate	Notes
XBee / ZigBee S2	2.1–3.6 V	UART	Level shift	3.3 V; 205 mA TX peak; sleep modes critical for battery
nRF24L01+	1.9–3.6 V	SPI	Level shift	130 nA sleep; power noise common issue — add 10 μ F + 100 nF caps
HC-05 Bluetooth	3.3–6 V	UART	Level shift (RX pin)	HC-05 RX is 3.3 V; use 1 k Ω +2 k Ω divider from 5 V TX
ESP8266	3.3 V	UART	Full level shift	500 mA peak; level shift ALL signal lines from 5 V MCU
LoRa SX1276	1.8–3.7 V	SPI	Level shift	3.3 V strictly; 120 mA TX; 3 km+ range at low data rate

Power management components

Device	Type	Max current	Notes
AMS1117-3.3 LDO	Linear	800 mA	1.2 V dropout; not for battery + high current
LM7805 LDO	Linear	1.5 A	Efficiency = $5/V_{in}$; heatsink above 500 mA
MP2307 / LM2596 Buck	Switching	–	92 % efficient; use from battery to MCU
TP4056	Li-Po charger	1 A	Charge + over-discharge + short-circuit protection
INA219	Current monitor	–	I ² C; 0.1 Ω shunt; reads current + voltage

Data storage and timekeeping

Device	Voltage	Interface	Intermediate	Notes
DS3231 RTC	2.3–5.5 V	I ² C	Direct	$\pm$ 2 ppm; CR2032 battery backup; address 0x68
AT24C256 EEPROM	1.7–5.5 V	I ² C	Direct	32 KB external; page write = 64 bytes; 5 ms write
MicroSD module	3.3 V (SPI)	SPI	Level shift	Most modules have 3.3 V reg + level shift built in; use FatFs
W25Q64 Flash	SPI 2.7–3.6 V	SPI	Level shift	8 MB; faster than SD for small writes; page = 256 B

Displays and input devices

Device	Voltage	Interface	Intermediate	Notes
HD44780 LCD	5 V	Parallel or I ² C	Level shift	Use I ² C backpack (PCF8574) to save GPIO pins
SSD1306 OLED	3.3–5 V	I ² C or SPI	Direct	1 KB framebuffer in SRAM; address 0x3C or 0x3D
TM1637 Segment	7- 3.3–5 V	2-wire	Direct	Custom 2-wire protocol; best for RPM/temperature display
Tactile switch	3.3–5 V	GPIO	Direct	Use internal pull-up; 10 ms software debounce
Rotary encoder	3.3–5 V	2× GPIO	Direct	A/B quadrature; interrupts on both pins for direction

Interface key summary

Interface	Key facts
I ² C	2-wire (SDA+SCL); needs 4.7 k Ω pull-ups; up to 127 addressed devices; half-duplex
SPI	4-wire (MOSI/MISO/SCK + 1 CS per device); full-duplex; fastest; no address
UART	2-wire (TX/RX); async; point-to-point; both sides must agree on baud rate
Analog	Direct voltage to ADC; check V _{cc} matches ADC reference
1-Wire	Single GPIO + 4.7 k Ω pull-up; multiple devices; parasitic power option